FORMADORES { IT }

Kubernetes

6. Contenedores como en Docker

Índice

El objeto container

Repaso de los parámetros que ya hemos visto

Variables de entorno

Links y conexión entre contenedores y pods

Entrypoint y command

Permisos y seguridad

Otros parámetros importantes

Objetivos

- Ser capaces de configurar contenedores como se hace con Docker
- Conocer todas las posibilidades del objeto container
- Ver cómo se conectan los contenedores entre sí
- Personalizar las opciones de seguridad

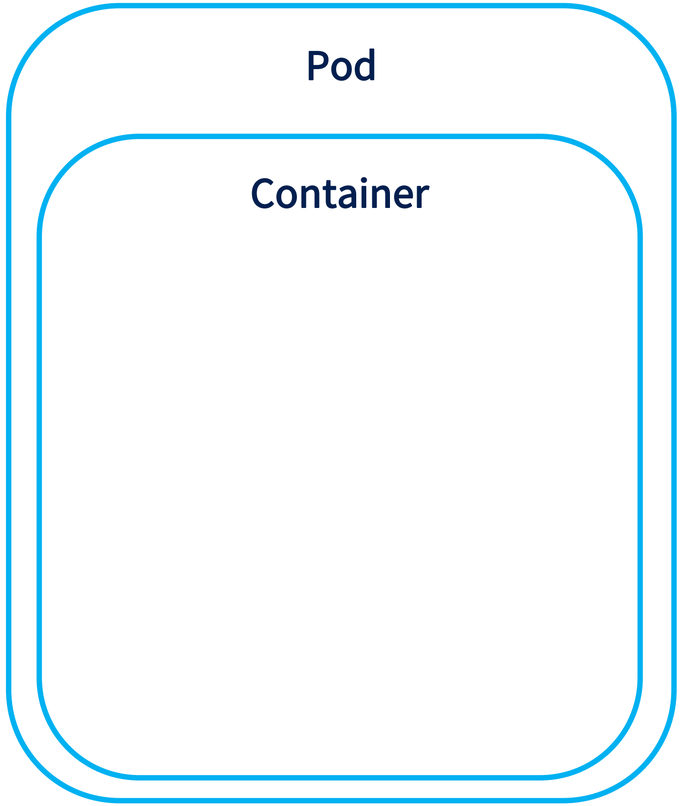
Mapa Conceptual

- Pods
  - Contenedores
    - Seguridad de contenedor
    - Variables de entorno
    - entrypoint y command
    - Otros parámetros del contenedor
  - Seguridad de pod
  - Otros parámetros del pod

El objeto container

Sabemos que los pods contienen entre sus especificaciones una lista de contenedores, pero por ahora no hemos visto cómo podemos configurar éstos como se puede hacer con Docker: personalizando variables de entorno, especificando el comando que queremos ejecutar en el contenedor, enlazándolo con otros contenedores, etc.

El objeto container, aunque no es un objeto nativo de Kubernetes y siempre irá dentro de un pod, ofrece todas estas posibilidades.



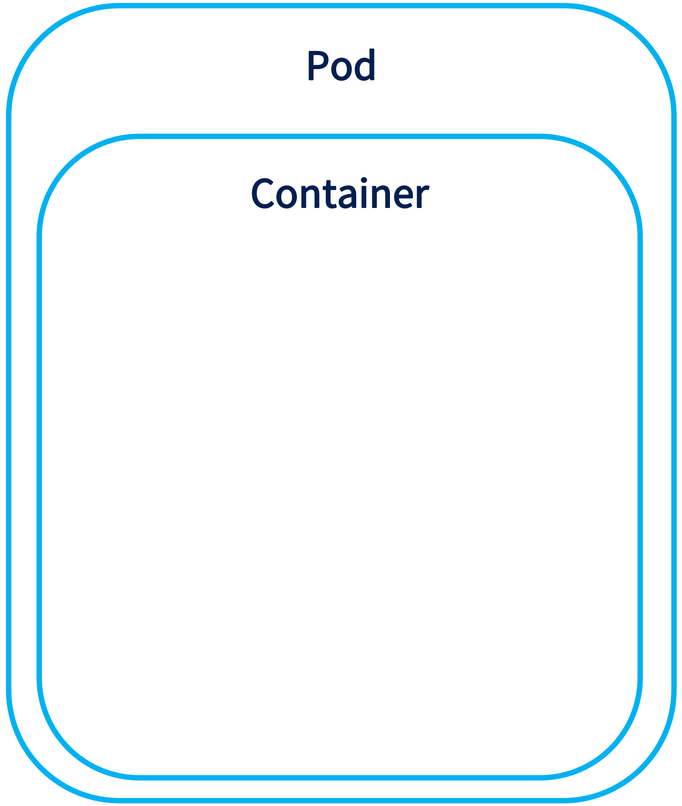
El objeto container

Para inspeccionarlas podemos utilizar el comando kubectl explain:

```
kubectl explain pod.containers
```

Como ves tiene muchas opciones. Algunas de ellas ya las hemos visto y en este módulo veremos las más importantes de las que quedan.

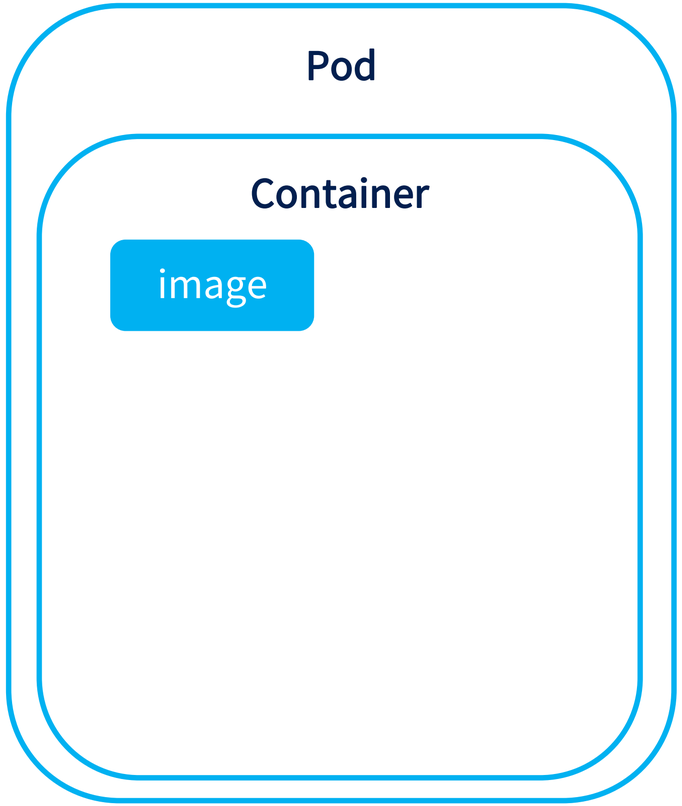
Conseguiremos ejecutar contenedores en Kubernetes con la misma (y mayor) flexibilidad y sencillez que en Docker



Los que ya conocemos

image

La imagen de Docker que va a ejecutar el contenedor.

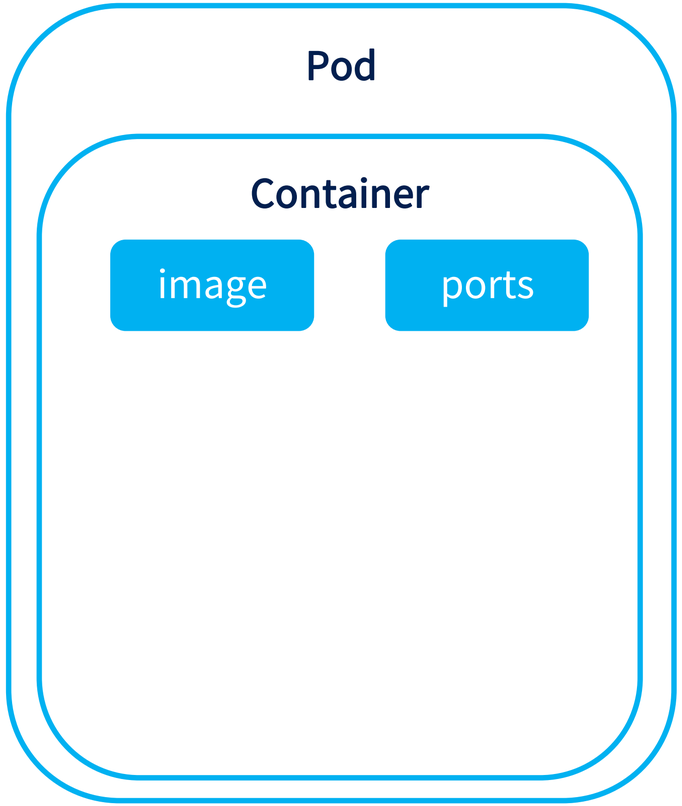


Los que ya conocemos

ports

Una lista de puertos y protocolos para exponer al exterior del contenedor.

Equivale a la opción de Docker --port / -p

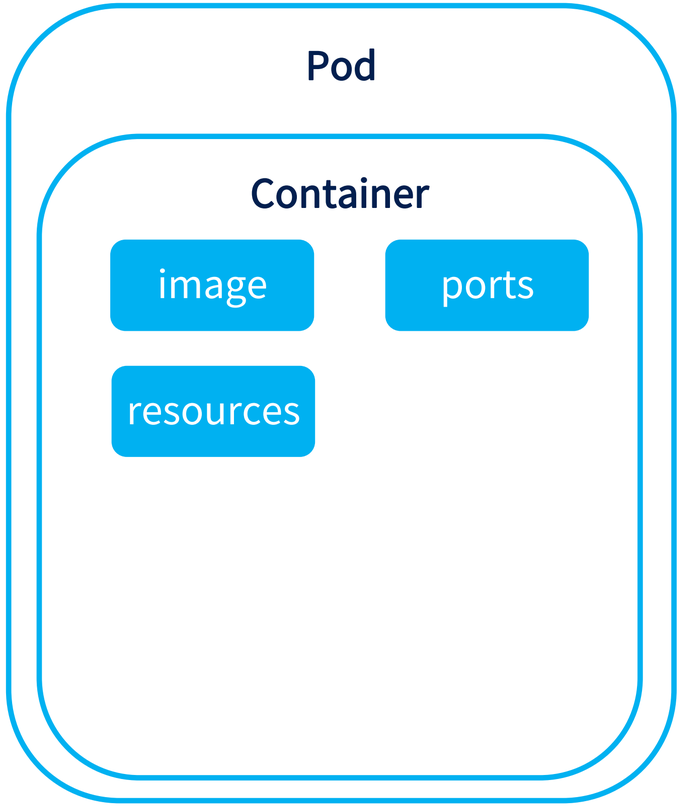


Los que ya conocemos

resources

Permite reservar y limitar recursos de CPU y memoria para cada contenedor.

Equivale a las opciones de Docker --memory, --memory-reservation, --cpu-quota

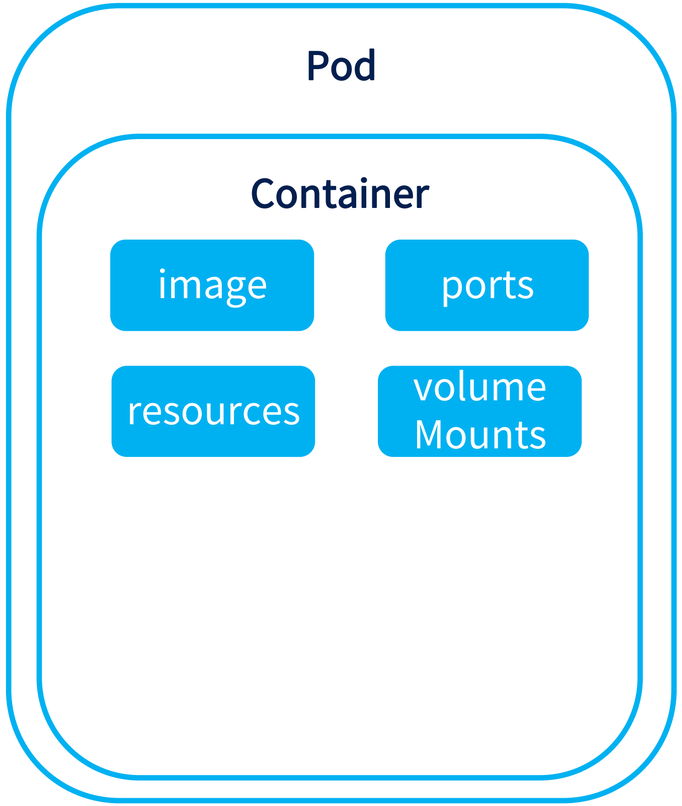


Los que ya conocemos

volumeMounts

Sirve para montar un volumen que esté ya configurado en el pod, dentro de la ruta que queramos.

Equivale a la opción de Docker --volumen / -v

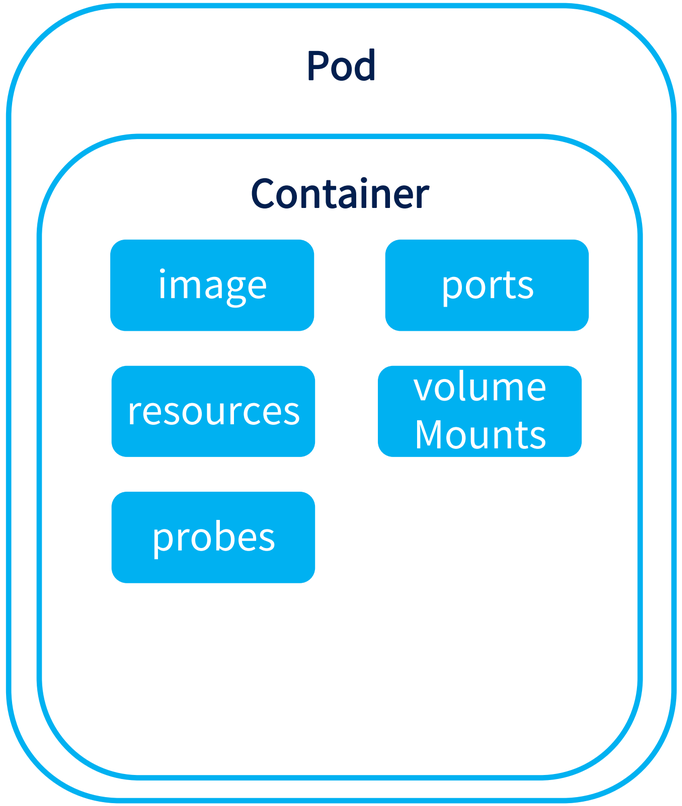


Los que ya conocemos

readinessProbe y livenessProbe

Con ellos creamos pruebas periódicas que hará Kubernetes para conocer el estado de salud del contenedor, y reiniciar el pod en caso de fallo.

Equivale a las opciones de Docker --health-.....

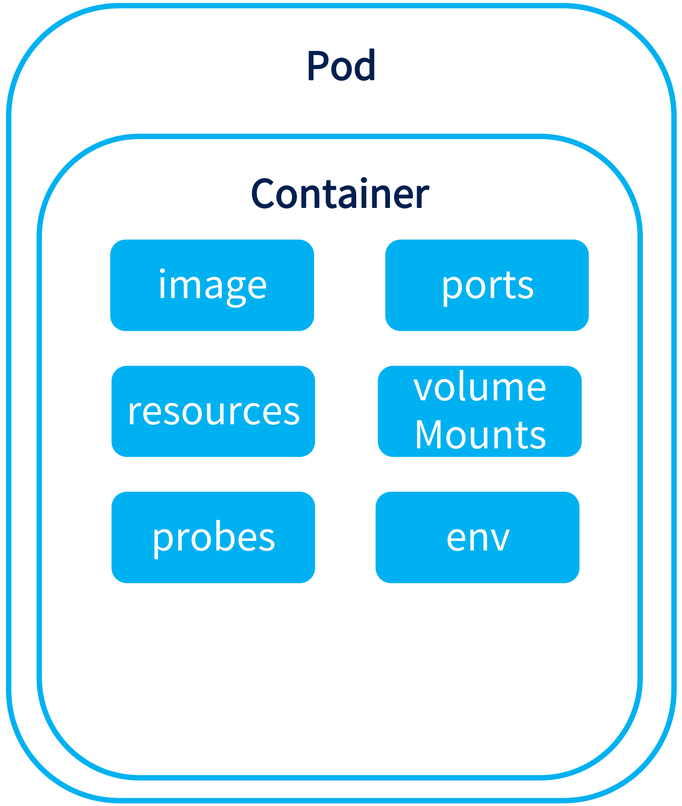


Variables de entorno

Configurar aplicaciones en contenedores con variables de entorno es una práctica habitual por su simplicidad.

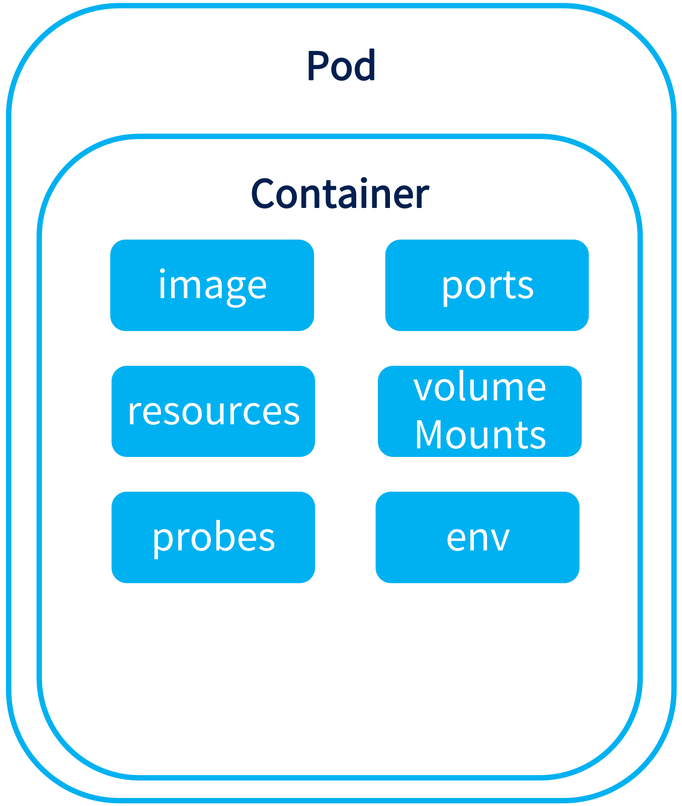
En Docker podíamos pasarle variables de entorno a los contenedores con la opción -e.

En Kubernetes también podemos incluir todas las variables de entorno en la configuración del contenedor, dentro del campo env



Variables de entorno

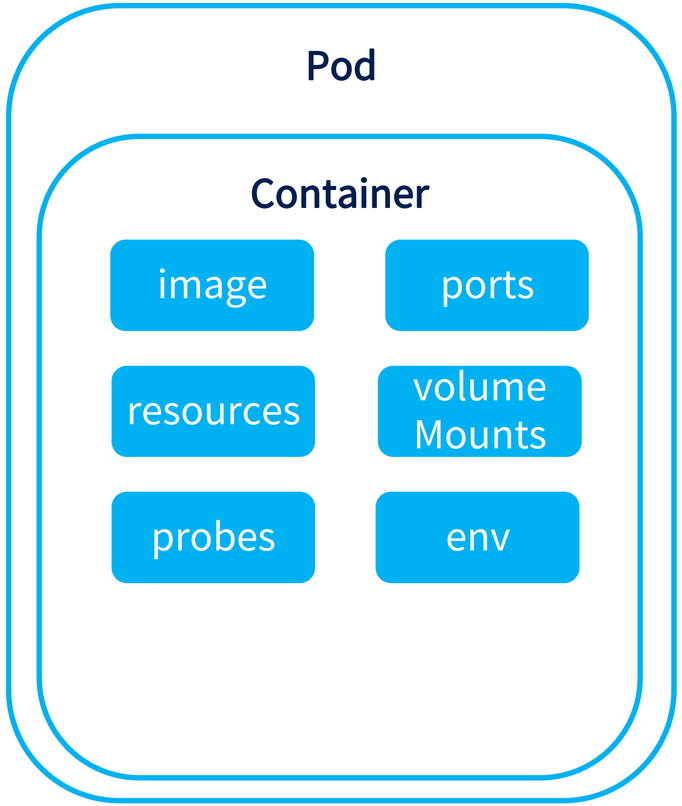
```
containers:
        - name: nginx
          image: nginx
          ports:
          - containerPort: 80
          env:
          - name: foo
            value: var
          - name: OTHER_VARIABLE
            value: OTHER_VALUE
```



Variables de entorno

Otra opción que nos ofrece Kubernetes es leer las variables de entorno de un ConfigMap o un secret.

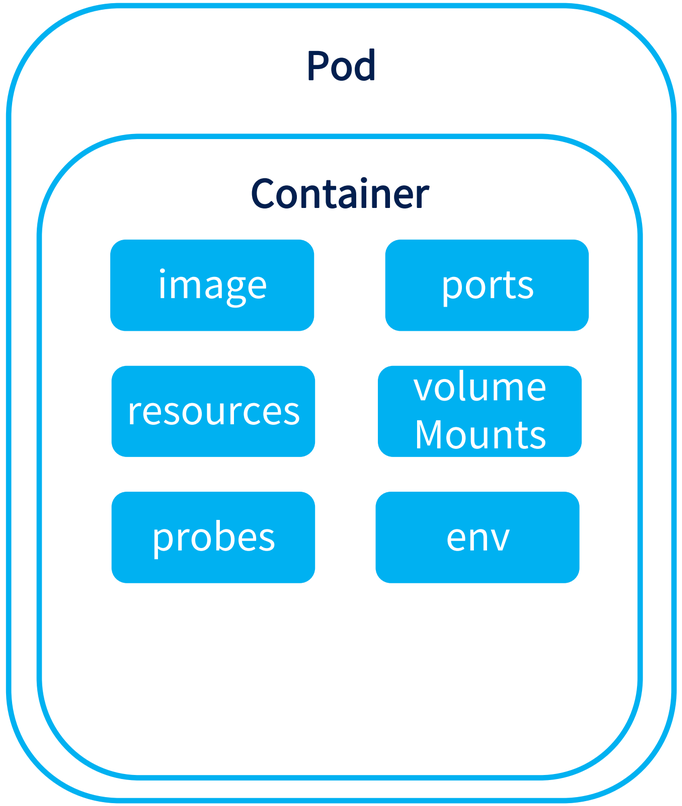
De esta forma como ya sabemos, podemos reutilizar el YAML del pod entre entornos, cambiando simplemente el nombre del ConfigMap del que leer las variables, en lugar de tener que cambiar todas en el mismo objeto del pod.



Variables de entorno

Para leer una única variable de entorno de un ConfigMap:

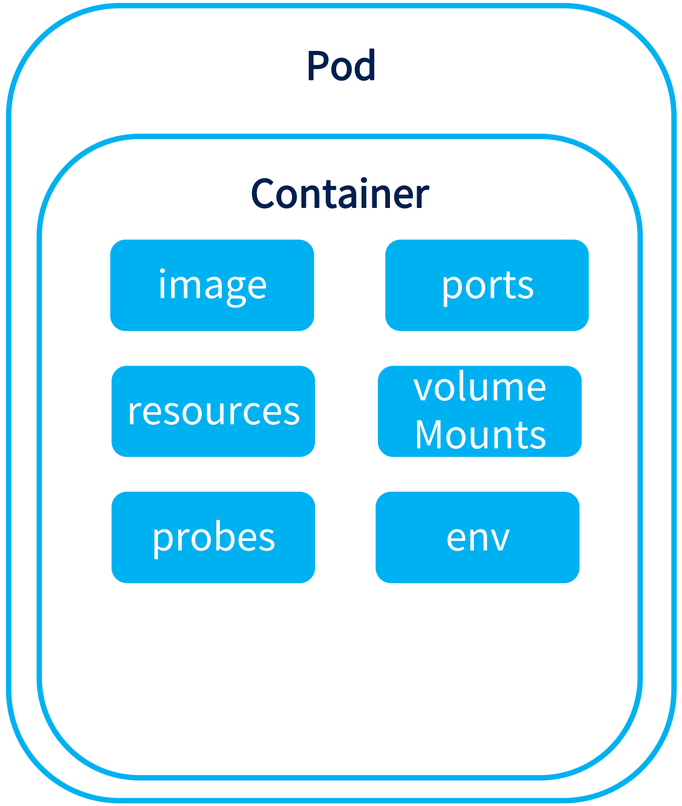
```
env:
- name: foo
  value: bar
- name: OTHER_VARIABLE
  value: OTHER_VALUE
- name: DB_HOST_FROM_CONFIGMAP
  valueFrom:
    configMapKeyRef:
      name: production-config
      key: DB_HOST
```



Variables de entorno

Para leer una única variable de entorno de un Secret:

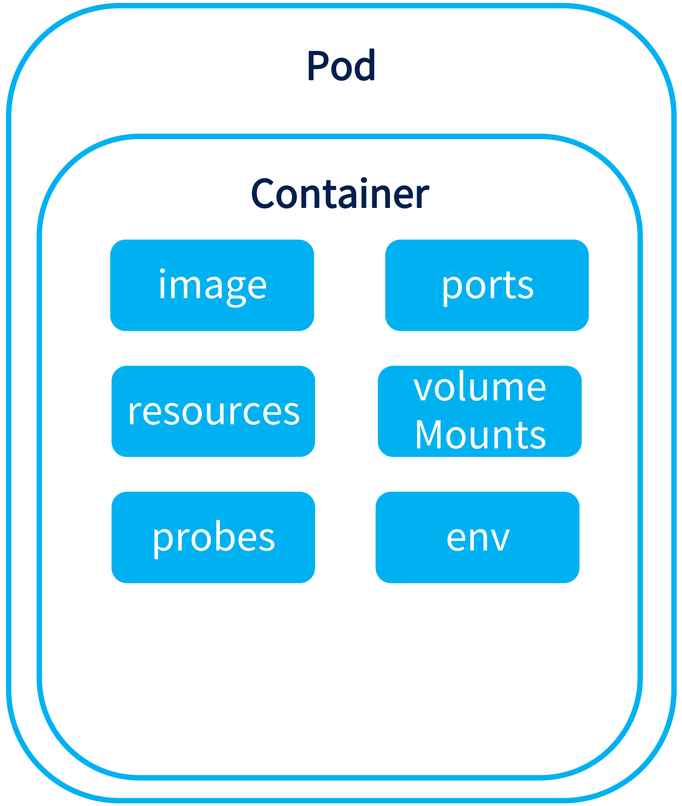
```
env:
- name: foo
  value: bar
- name: OTHER_VARIABLE
  value: OTHER_VALUE
- name: DB_PASSWORD_FROM_SECRET
  valueFrom:
    secretKeyRef:
      name: production-credentials
      key: DB_PASSWORD
```



Variables de entorno

Para importar como variables de entorno todas las variables de un ConfigMap

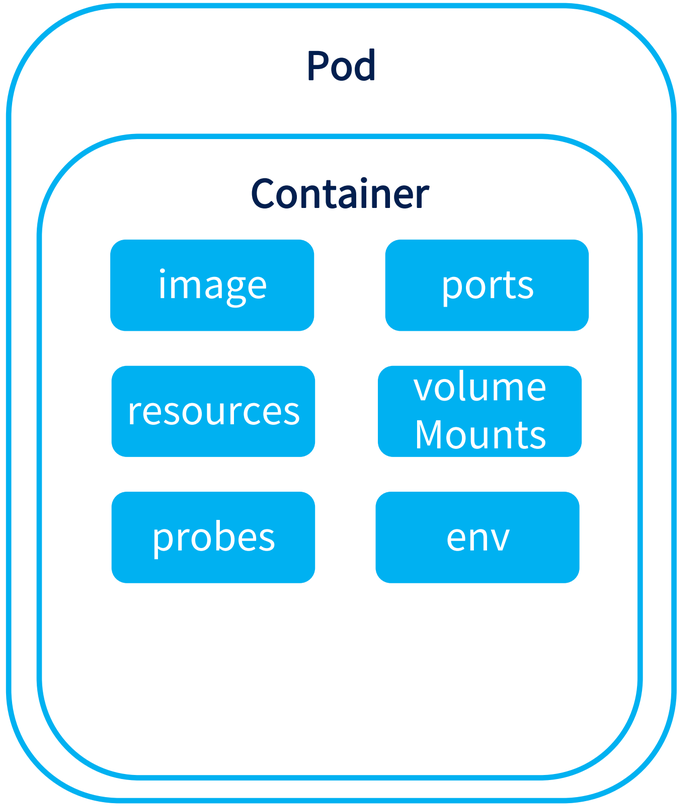
```
containers:
        - name: nginx
          image: nginx
          ports:
          - containerPort: 80
          envFrom:
          - configMapRef:
              name: production-config
```



Variables de entorno

Para importar como variables de entorno todas las variables de un Secret:

```
containers:
        - name: nginx
          image: nginx
          ports:
          - containerPort: 80
          envFrom:
          - configMapRef:
              name: production-config
          - secretRef:
              name: production-credentials
```



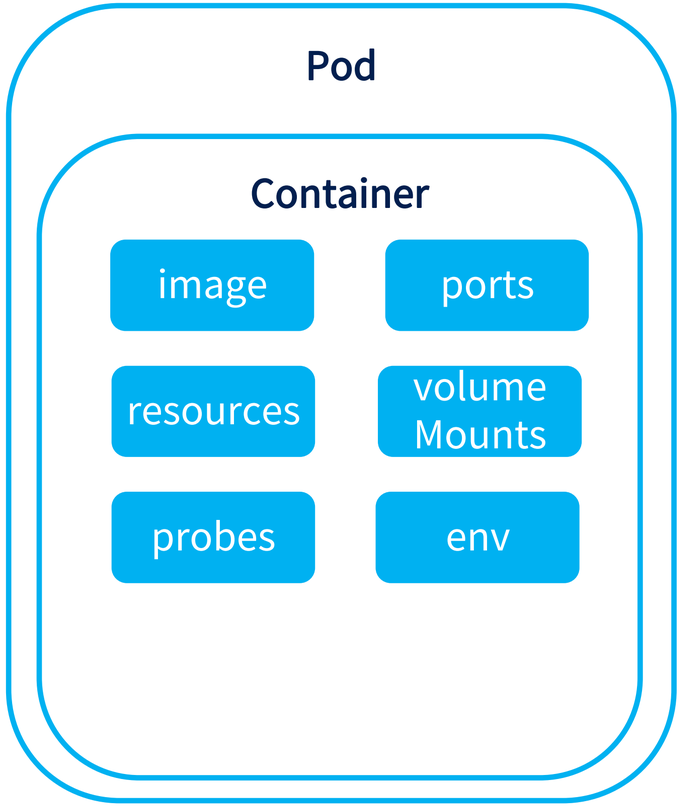
Variables de entorno

Vamos a probar estas variables en el entorno de prueba. Crea el Deployment y el Service que hemos visto en módulos anteriores y prueba a añadir algunas variables de entorno.

Crea también un ConfigMap y un Secret para leer otras variables de ellos.

Una vez desplegado todo, vamos a abrir un terminal dentro de un contenedor para inspeccionar estas variables:

```
kubectl exec -ti <POD NAME> /bin/bash
```



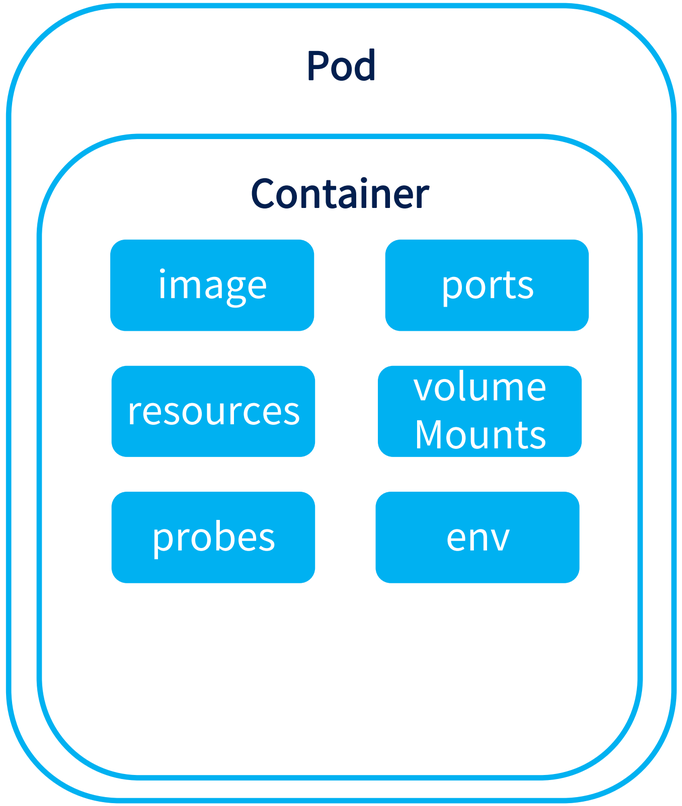
Variables de entorno

Con el comando env listamos todas las variables que están configuradas en el contenedor.

Aparte de todas las que hayas creado tú mismo, Kubernetes nos añade muchas otras de la forma

<SERVICE_NAME>_SERVICE_HOST
<SERVICE_NAME>_PORT_....

Estas variables están por defecto en todos los contenedores, nos ayudan a conectarnos más fácilmente a otros pods mediante sus services. Utilizándolas no necesitamos conocer ninguna IP, nos basta con saber el nombre para encontrar la variable en cuestión.



Conexión entre contenedores

Las variables de entorno que crea Kubernetes por defecto nos recuerdan inevitablemente a las que creaba Docker cuando enlazas contenedores con la opción --link / -l

En Kubernetes no es necesario especificar enlaces entre contenedores o pods. **Todos los pods de Kubernetes están enlazados entre sí** y son accesibles por defecto a través de su nombre.

Si tenemos un pod con un servidor web y otro con su base de datos, no necesitamos especificar nada para que se conecten entre sí. Tampoco entre contenedores del mismo pod.

Conexión entre contenedores

Los contenedores **dentro del mismo pod** se ejecutan dentro del mismo espacio de red. Esto significa que a nivel de red actúan como procesos que se están ejecutando en la misma máquina, por lo que son accesibles mediante la dirección local: **localhost** o 127.0.0.1

Esto también significa que los contenedores dentro del mismo pod confían entre sí y no hay barreras a nivel de red para limitar el acceso entre ellos.

Es una buena práctica tratar de separar diferentes servicios en diferentes pods. Ayudará a la seguridad y escalabilidad de la aplicación.

Conexión entre contenedores

Los contenedores **en distintos pods** también son accesibles entre sí por defecto, siempre a través de los puertos especificados en ports.

Para conectarse a otros pods, se puede simplemente utilizar la IP privada del pod o de un service, siempre serán accesibles desde cualquier otro pod.

Como las IPs privadas son efímeras y cambiarán siempre que se recree el recurso, Kubernetes nos da otros métodos para conectarnos a los recursos simplemente sabiendo el nombre y el namespace.

Conexión entre contenedores

El primer método sería utilizando las **variables de entorno** que nos da Kubernetes en todos los pod por defecto.

Con ellas tenemos acceso a todas las IPs privadas y puertos de los services en nuestro namespace.

Este no es el mejor método para conectarse entre distintos pods ya que en caso de que esas IPs o puertos cambien, las variables de entorno no son actualizadas hasta que se reinicie el pod.

Conexión entre contenedores

El otro método y el más efectivo es utilizando las **direcciones DNS** que nos da Kubernetes.

Kubernetes tiene un servicio de **DNS automático para todos los pods y services**. Simplemente con su nombre y namespace, podemos encontrar la dirección IP de cada recurso.

Para ello no es necesario configurar nada ni crear links como en Docker, se crean todas por defecto.

Conexión entre contenedores

<SERVICE_NAME>.<NAMESPACE>.svc.cluster.local

<POD_NAME>.<NAMESPACE>.pod.cluster.local

Realmente, sólo deberías utilizar la primera. No deberías conectarte directamente a ningún pod en concreto, ya que éstos son efímeros, y para eso está el objeto service.

Conexión entre contenedores

Además, dentro del mismo namespace puedes conectarte simplemente con el nombre del service, el pod está configurado para probar las direcciones dentro del mismo namespace automáticamente solo con el nombre.

Incluso siempre que estés dentro de un pod en el mismo clúster de Kubernetes, puedes ignorar también la parte de svc.cluster.local, y utilizar sólo

<SERVICE_NAME>.<NAMESPACE>

Conexión entre contenedores

¿Cómo funciona este servicio DNS automático?

Kubernetes configura el archivo resolv.conf en todos los contenedores de forma automática, para que se conecten a su servicio DNS, y por defecto pruebe los dominios que hemos visto.

El servidor DNS también está en el propio clúster, en el namespace kube-system. Son los pods con nombre coredns (en otros cluster verás también kube-dns). Y el service del mismo nombre.

Verás que el service tiene precisamente la IP que está configurada en el archivo resolv.conf. Ésta es una IP fija de cada clúster que no se puede alterar.

Conexión entre contenedores

En resumen, este es el sistema de conexión entre pods y de **service discovery** que nos da Kubernetes.

En otras plataformas, configurar un DNS automático y una red para conectar contenedores en distintos nodos nos llevaría a mantener sistemas complejos y difíciles de poner en marcha.

Ésta es una de las principales ventajas de usar Kubernetes, tener configurada automáticamente y por defecto la red entre todos los contenedores de todos los nodos con service discovery incluido.

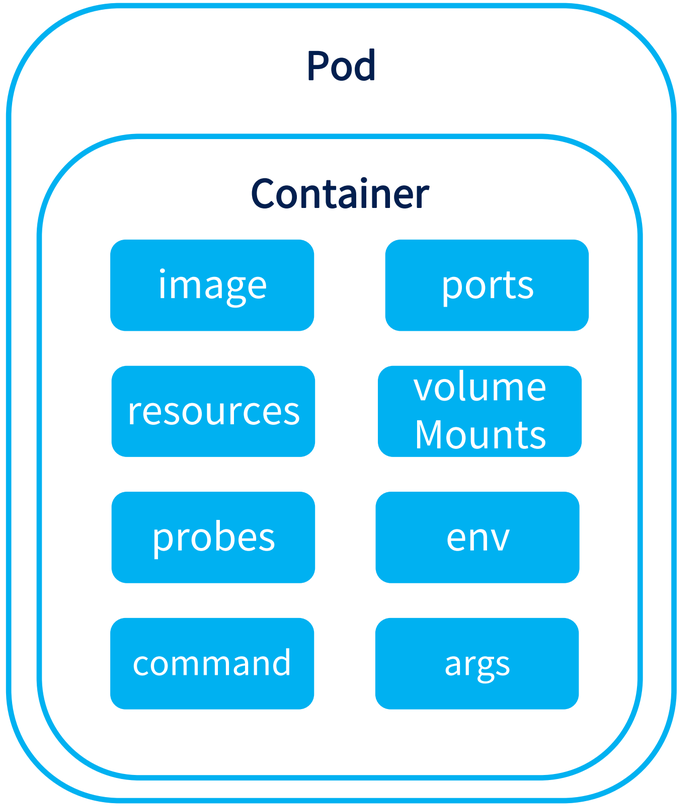
command y args

En Docker podíamos especificar el comando que queríamos ejecutar al levantar un contenedor, simplemente añadiéndolo después de la imagen en docker run.

Este comando se pasaba como parámetro al entrypoint del contenedor.

Incluso podíamos modificar el entrypoint del propio contenedor utilizando

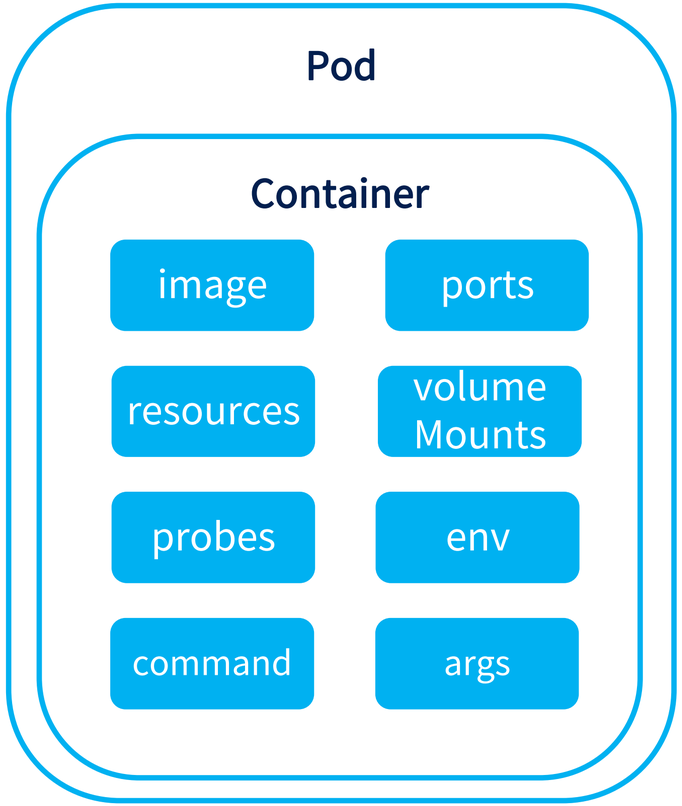
En Kubernetes tenemos exactamente la misma opción, utilizando dos parámetros del contenedor: command y args



command y args

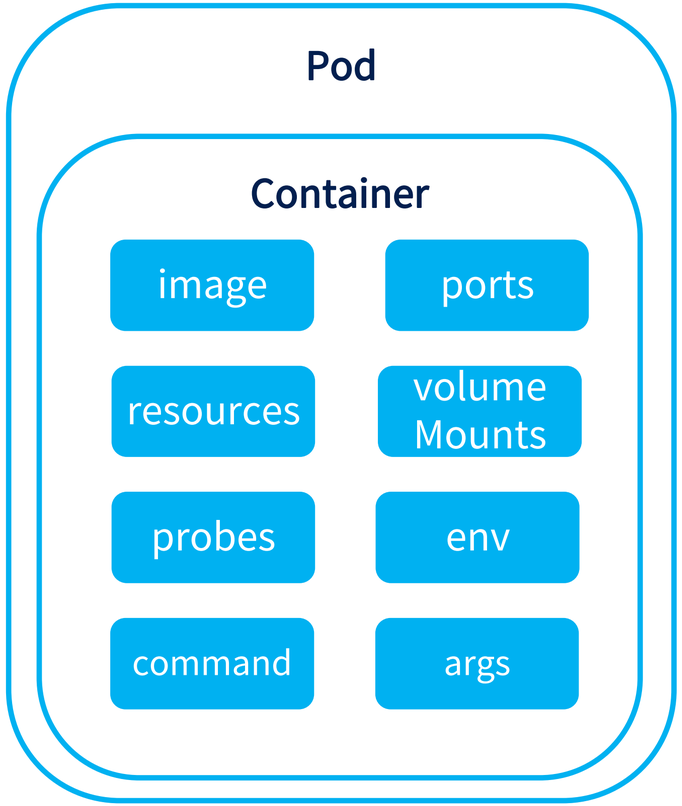
Al principio puede parecer poco intuitivo pero la equivalencia con Docker es esta:

- command representa el entrypoint del contenedor. En general, este no debe ser alterado, y solemos dejar el entrypoint por defecto de cada contenedor.
- args representa lo que llamamos CMD o comando en Docker. Son los argumentos que le pasamos al entrypoint, y que personalizan la forma en la que se ejecuta éste.



command y args

```
containers:
        - name: ubuntu
          image: ubuntu
          ports:
          - containerPort: 80
          command:
            - /bin/bash
          args:
            - -c
            - "echo Hello World"
```

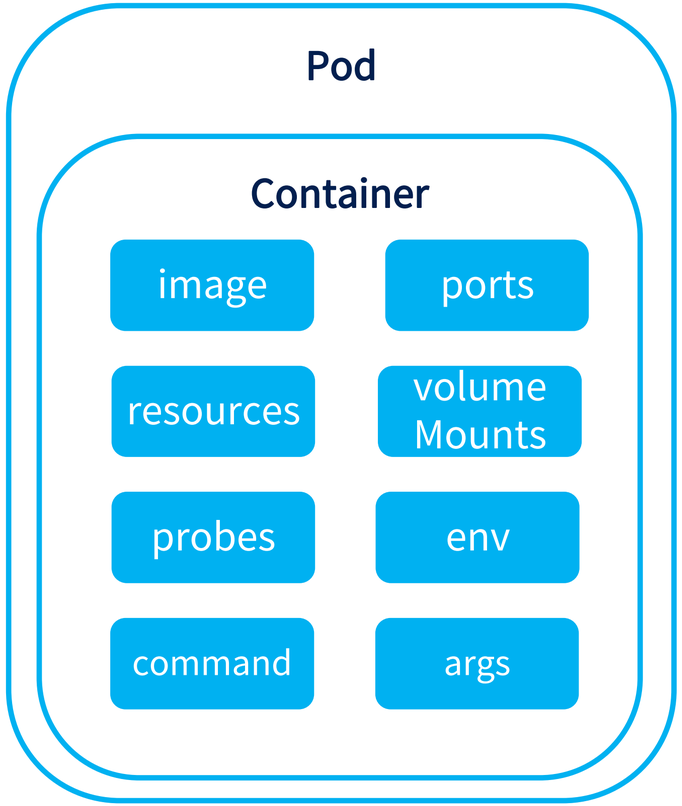


command y args

También puedes encontrarlo en la forma:

```
containers:
        - name: ubuntu
          image: ubuntu
          ports:
          - containerPort: 80
          command: ["/bin/bash"]
          args: ["-c", "echo Hello World"]
```

Son equivalentes

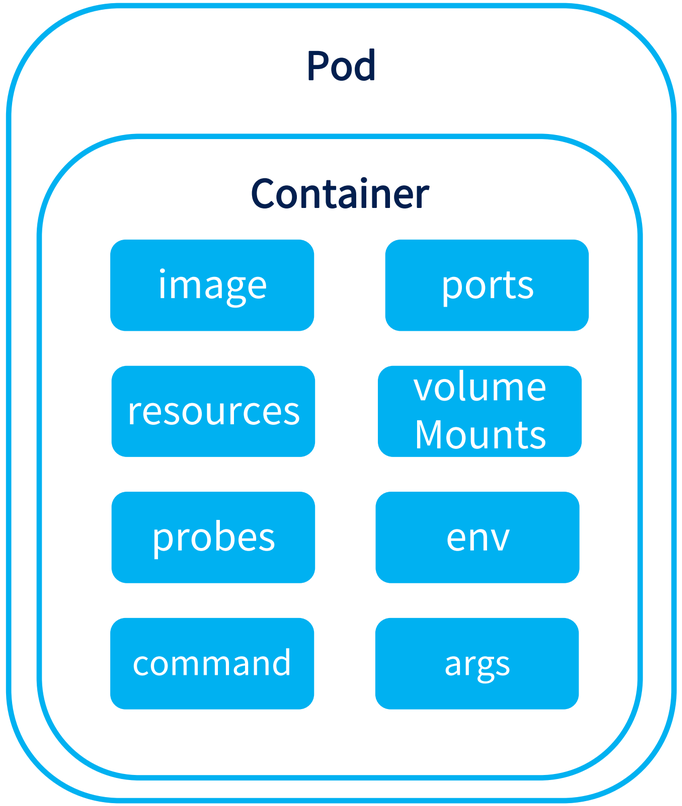


command y args

Tanto command como args son **arrays**. Cada elemento del array representa un argumento distinto. Este comportamiento es similar al que tenemos en Dockerfile cuando definimos los mismos parámetros.

En el command, cuando necesitemos utilizarlo, el primer elemento del array será el binario que se va a ejecutar en el contenedor. Los siguientes serán los argumentos.

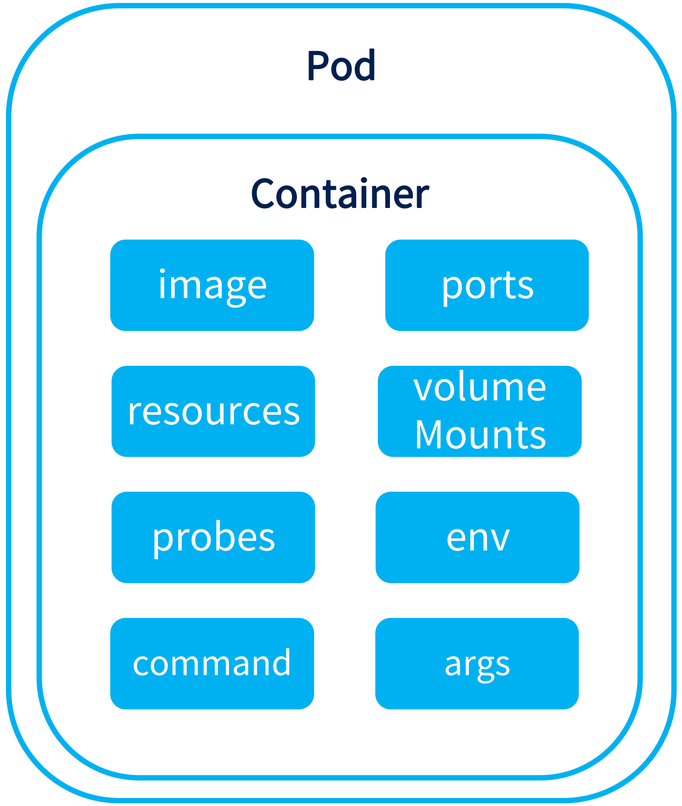
En args, todos los elementos del array se añadirán a los argumentos del entrypoint.



command y args

Este es un ejemplo típico en el que modificamos el entrypoint (command) y los argumentos o CMD (args) para que en lugar de ejecutar el binario para el que está pensado el contenedor, ejecutamos un comando en bash que nos permitirá debuggear sin que se reinicie el pod constantemente:

```
command:
  - /bin/bash
args:
  - -c
  - "sleep 3600"
```



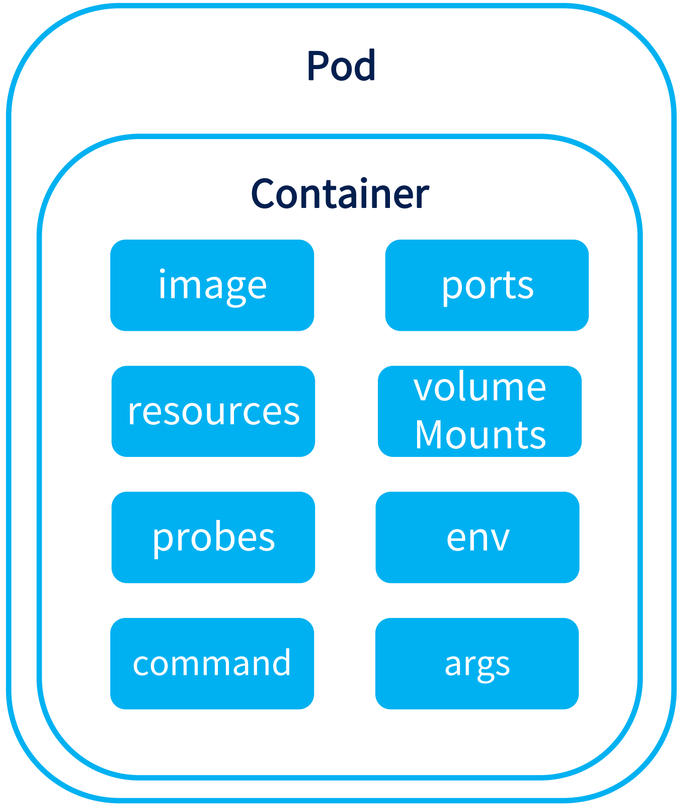
command y args

Vamos a probar a añadir estos argumentos al contenedor de nginx:

nginx-debug -g 'daemon-off'

Según la documentación de la imagen de Docker de Nginx, con ellos ejecutamos nginx-debug en lugar del nginx corriente, que nos dará logs más detallados de la ejecución.

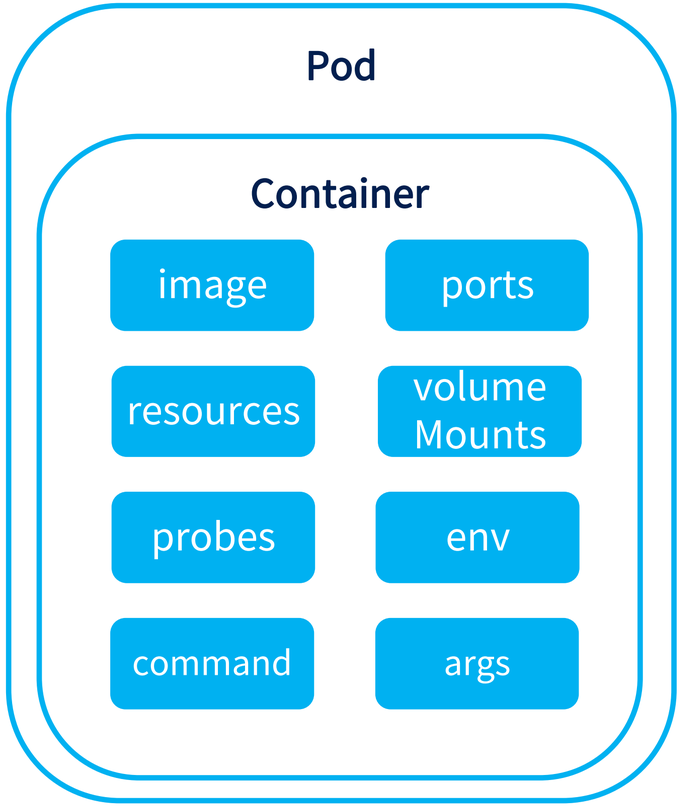
```
args:
- nginx-debug
- -g
- daemon off;
```



command y args

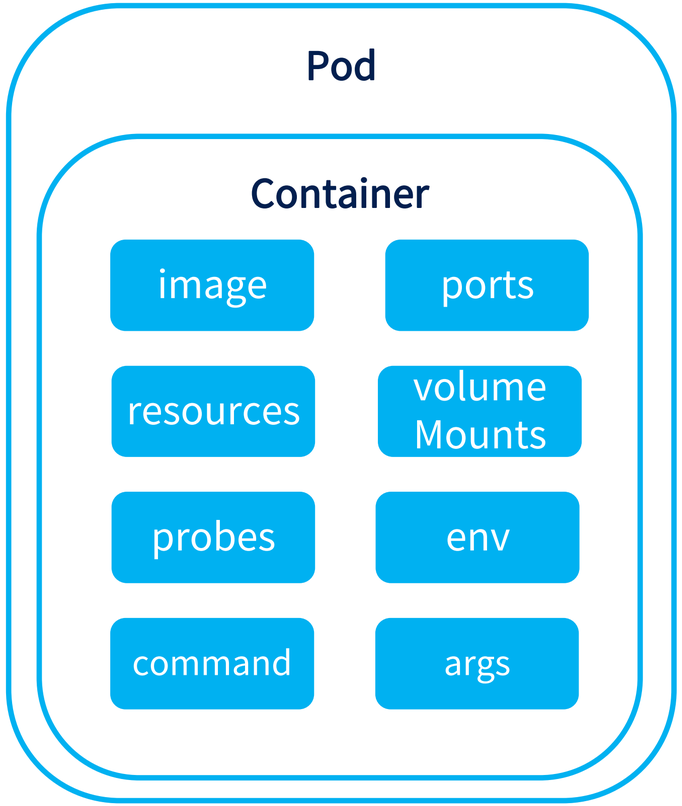
También vamos a probar a sobrescribir el entrypoint por defecto para probar el parámetro command, y pasarle como argumentos un script de bash personalizado:

```
command:
  - /bin/bash
args:
  - -c
  - "while true; do date; sleep 5; done"
```



command y args

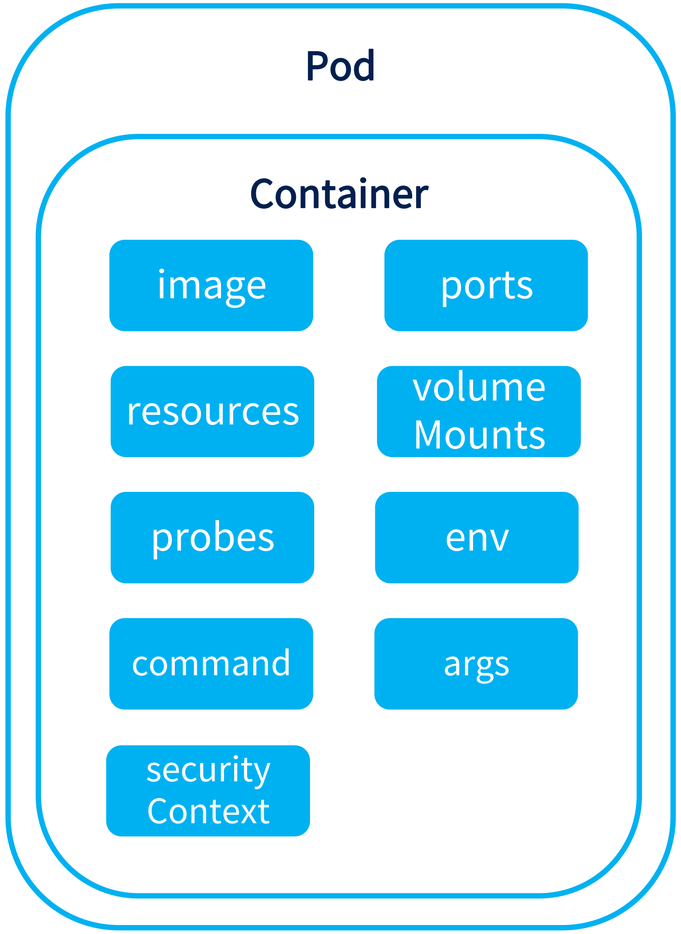
En este caso el contenedor no ejecutará el servidor de Nginx que ejecuta por defecto, sino que simplemente imprimirá la fecha y hora cada 3 segundos en un bucle infinito.



Security context

Dentro de este parámetro se agrupan varias opciones de seguridad de ejecución del contenedor:

1. privileged
2. runAsUser
3. runAsGroup
4. readOnlyRootFilesystem
5. runAsNonRoot
6. capabilities
7. seLinuxOptions
8. …

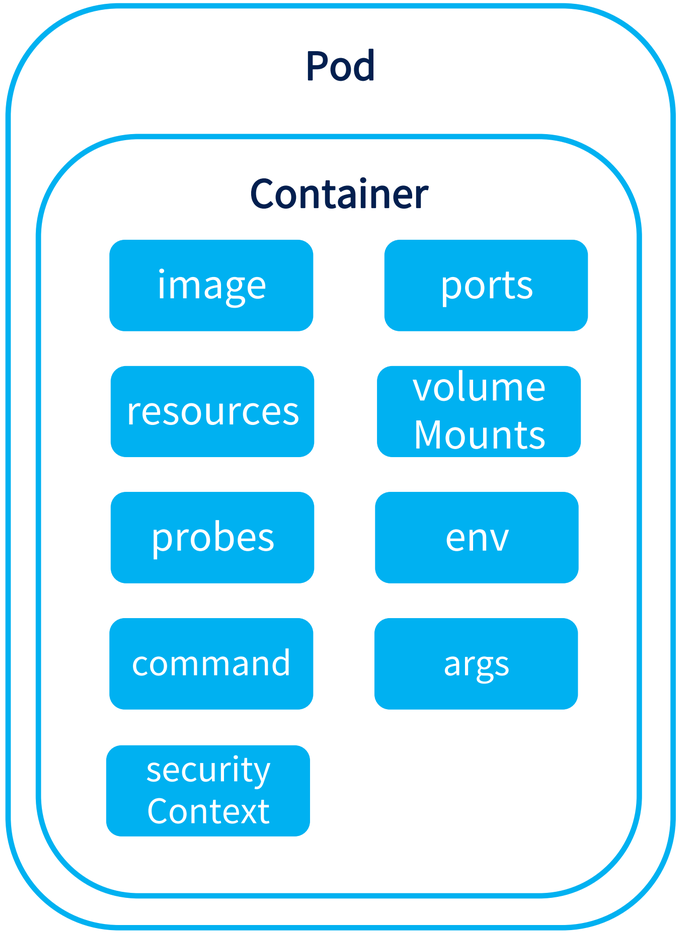


Security context

Especialmente útiles son las opciones runAsUser y runAsGroup, que nos permiten modificar el usuario y el grupo con el que se ejecutan por defecto el contenedor. Por ejemplo podemos sobrescribir el usuario root de muchos contenedores para que sean más seguros.

El parámetro privileged hace la misma función que --privileged en docker run, y ejecutará el contenedor sin ninguna medida de seguridad, con todos los permisos para interactuar con el host.

Hay que tener cuidado con las imágenes que ejecutamos con estos permisos, pero hay casos en los que es necesario. Por ejemplo, el kube-proxy necesita permisos para modificar iptables en cada host, y lo hace desde dentro de un contenedor.

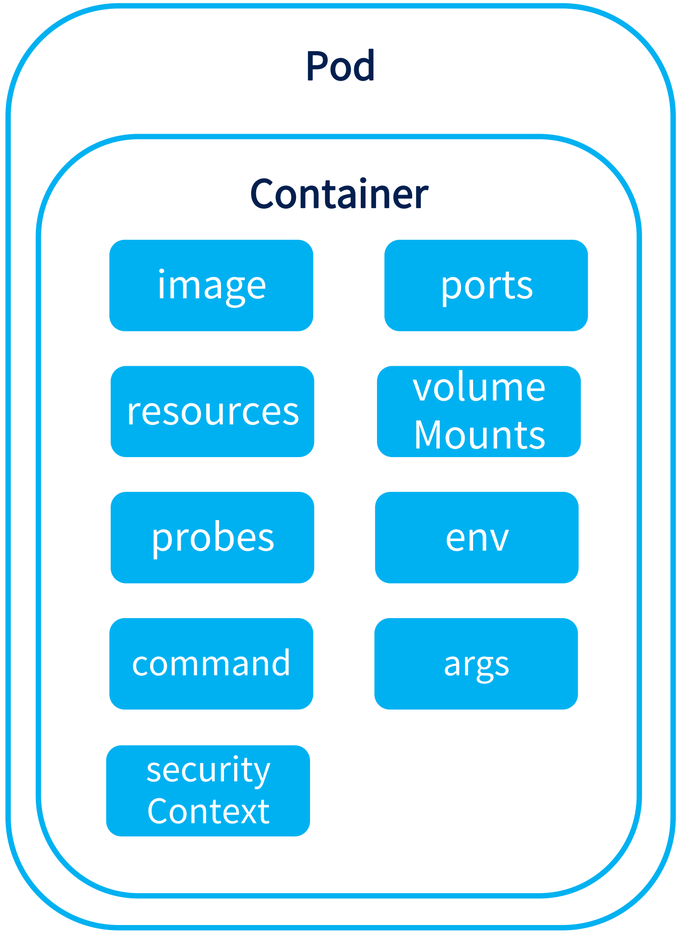


Security context

Los demás parámetros son más avanzados pero nos permiten prevenir imágenes maliciosas que quieran escalar privilegios, o también dar permisos mucho más específicos que con privileged.

Esto lo podemos hacer con el apartado capabilities o con seLinuxOptions, pudiendo añadir por ejemplo sólo permisos de red para un contenedor en concreto, en lugar de darlo todo el control del host.

Es suficiente con saber que estos parámetros existen y que dentro de Kubernetes podemos conseguir un nivel de seguridad similar a cualquier otro sistema, de contenedores o nativo.

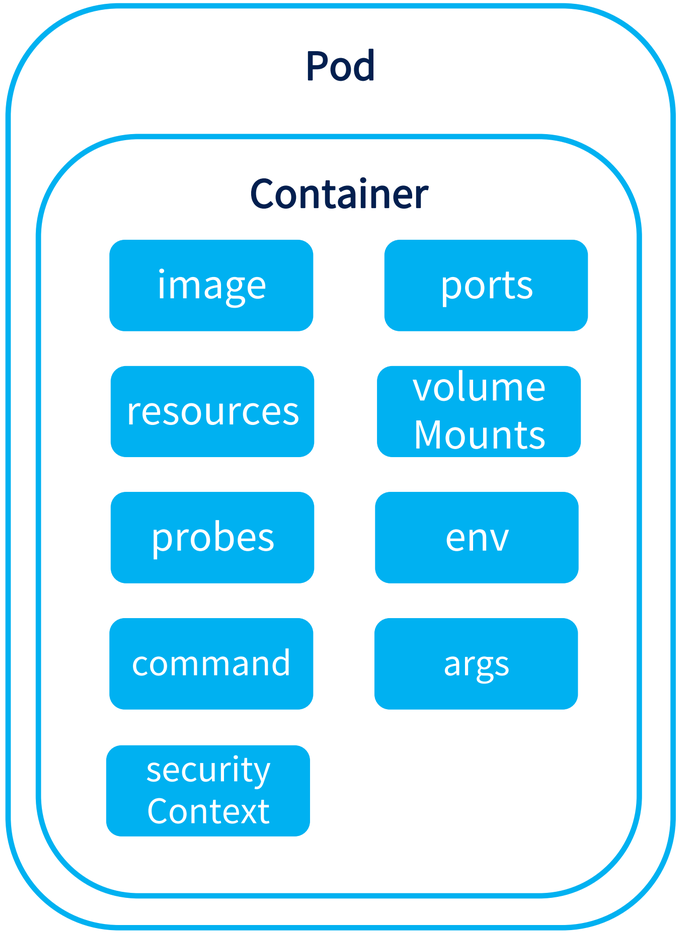


Otros parámetros de contendor

imagePullPolicy: Permite especificar la política para descargar las imágenes de Docker (hacerlo siempre que se levante un pod nuevo, o saltarse ese paso si la imagen ya está presente en el host para ganar velocidad).

workingDir: Puede modificar el directorio por defecto en el que se ejecuta el proceso del contenedor. Hace la misma función que el parámetro similar del Dockerfile o de docker run.

lifecycle: Dentro de este parámetro podemos añadir scripts que se ejecuten al inicio o al final del ciclo de vida del pod.

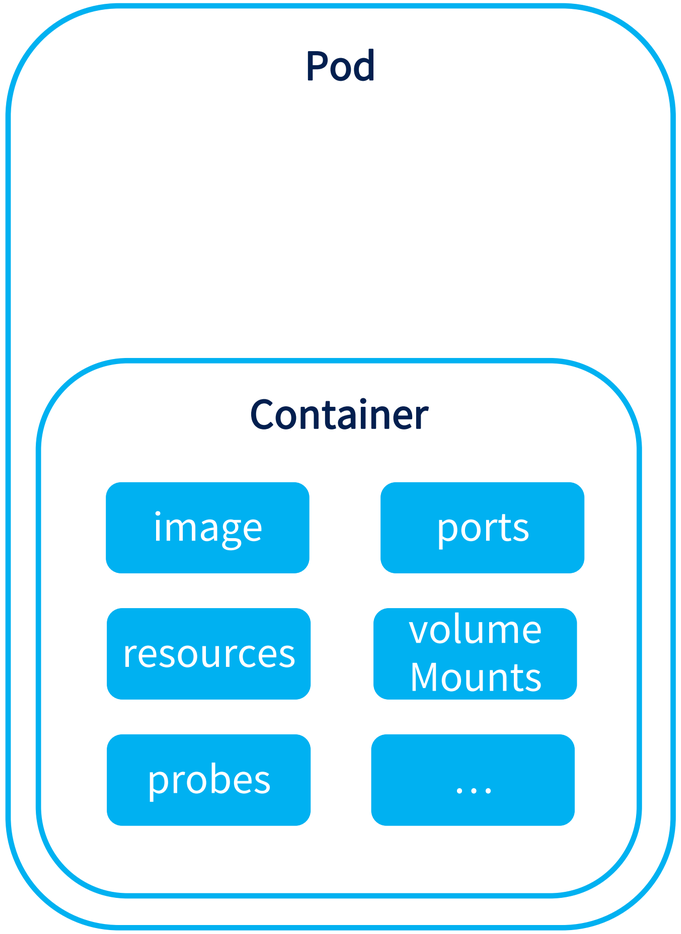


Parámetros del pod

Hay opciones importantes que no se aplican cada contenedor por separado, sino que se aplican al objeto pod y afecta a todos los contenedores que forman parte de él.

En Docker sí que había opción de aplicarlos por separado a cada contenedor pero en Kubernetes los contenedores del mismo pod tienen cosas en común, como el espacio de red, que hace que estos parámetros tengan que ser comunes a todos los contenedores.

Veremos algunos de los más relevantes.

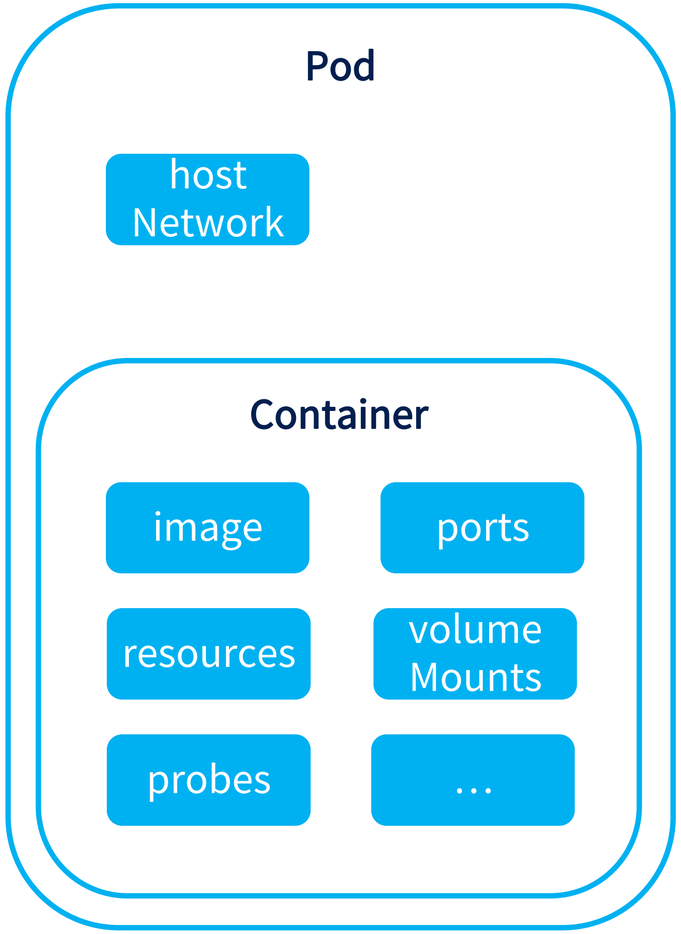


hostNetwork

También podemos ejecutar el pod en el mismo espacio de red que el host, de forma que tiene acceso a sus puertos y tiene los mismos permisos de red que el nodo en el que se está ejecutando:

hostNetwork: true

Al igual que opciones de securityContext, implica muchos agujeros de seguridad que debemos cuidar.



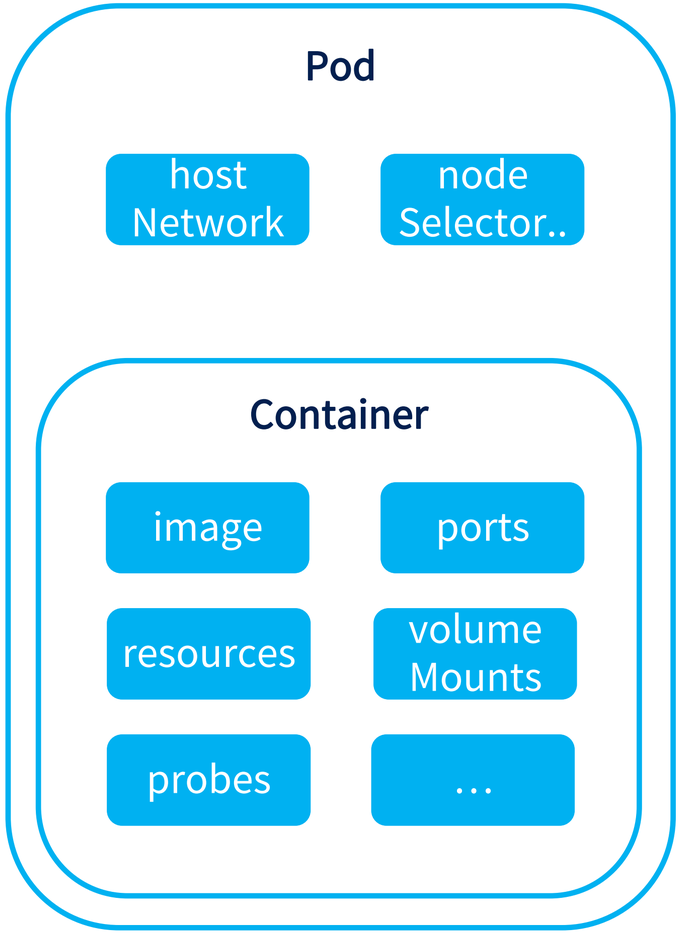
Controlar el nodo del pod

Hay varios mecanismos para controlar y limitar los nodos en los que se puede ejecutar un pod.

Los parámetros más relevantes son:

- nodeSelector
- tolerations
- affinity

No vamos a verlos en este curso, pero es importante saber que existe esa posibilidad y conocer al menos el nombre de los parámetros que podemos utilizar para ello.



¡Fin del módulo!

En este módulo hemos conocido todas las opciones de configuración de contenedores de Kubernetes:

- Variables de entorno
- A leerlas a partir de un ConfigMap o un secret.
- Conexión entre contenedores y pods. DNS interno
- Entrypoint y comando
- Varias opciones de seguridad
- Parámetros a configurar a nivel de pod

Extras

Páginas de la documentación oficial:

Variables de entorno
Variables de entorno desde secrets
DNS en Kubernetes
Command y args
securityContext

Referencia del objeto contenedor
Referencia del objeto pod

¡Gracias!